Homework 07

Use either R or Python.

Question 1: LDA

Let's start by estimating the components of LDA. From the lecture notes, for each target point $\mathbf{x}_0$, we find the class label k with the largest score

$$\mathbf{x}_0^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \log \pi_k.$$

Thus, we need to estimate:

- the prior probabilities π_k;
- the mean vector (centroid) μ_k for each class; and
- the pooled covariance matrix Σ.

Use saheart.csv, the `SAheart` data from `ElemStatLearn`. The two classes are defined by `chd`, the coronary heart disease indicator. Use all nine other variables as numerical predictors, coding `famhist` as 1 for `Absent` and 2 for `Present`. The following code constructs an $n \times 9$ predictor matrix $\mathbf{X}$ and an n-entry response vector $\mathbf{y}$, where n is the number of observations.

R

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd
```

a. Calculate the prior probabilities, class means, and pooled covariance matrix from these data. Use the pooled covariance denominator $n - K$ from the lecture, where $K = 2$ is the number of classes. ::: {.panel-tabset .sync-code-panels group="language"} ### R

```
n <- nrow(X); p <- ncol(X); K <- 2
classes <- c(0, 1)

pi_hat <- sapply(classes, function(k) mean(y == k))
mu_hat <- sapply(classes, function(k) colMeans(X[y == k, ]))  # p x K

Sigma_hat <- matrix(0, p, p)
for (k in classes) {
  Xc <- sweep(X[y == k, ], 2, mu_hat[, k + 1])
  Sigma_hat <- Sigma_hat + t(Xc) %*% Xc
}
Sigma_hat <- Sigma_hat / (n - K)

pi_hat
```

```
[1] 0.6536797 0.3463203
```

```
mu_hat
```

```
               [,1]       [,2]
sbp       135.460265 143.737500
tobacco     2.634735   5.524875
ldl         4.344238   5.487938
adiposity  23.969106  28.120250
famhist     1.317881   1.600000
typea      52.367550  54.493750
obesity    25.737450  26.622937
alcohol    15.931358  19.145250
age        38.854305  50.293750
```

```
Sigma_hat
```

```
                 sbp    tobacco         ldl  adiposity    famhist        typea
sbp       405.4347786 14.5851882  4.58124834 49.1643582 0.33698819 -15.58772153
tobacco    14.5851882 19.2425564  0.76320271  7.5381093 0.01582050  -2.05730592
ldl         4.5812483  0.7632027  4.00058129  6.0326970 0.09185024   0.34459233
adiposity  49.1643582  7.5381093  6.03269701 56.7528931 0.43280832  -5.30956633
famhist     0.3369882  0.0158205  0.09185024  0.4328083 0.22583357   0.08112007
typea     -15.5877215 -2.0573059  0.34459233 -5.3095663 0.08112007  95.56564291
obesity    18.9388430  1.8334506  2.66005159 22.7078852 0.18402776   2.64006728
alcohol    64.4003517 20.5170451 -2.53290675 16.1190253 0.76846885   7.96003164
age        95.1338895 22.7654113  6.47892049 60.5086715 0.99736251 -20.27833492
              obesity     alcohol         age
sbp       18.9388430  64.4003517  95.1338895
tobacco    1.8334506  20.5170451  22.7654113
ldl        2.6600516  -2.5329068   6.4789205
adiposity 22.7078852  16.1190253  60.5086715
famhist    0.1840278   0.7684688   0.9973625
typea      2.6400673   7.9600316 -20.2783349
obesity   17.6154236   4.6893563  15.6969995
alcohol    4.6893563 598.2766133  27.8859413
age       15.6969995  27.8859413 184.1321373
```

:::

The priors are the class proportions, the means are the 9-dimensional class centroids (a 9×2 matrix), and $\hat{\Sigma}$ is the 9×9 pooled covariance matrix.

b. Use your estimated values to predict the label of each observation in the training data. These are in-sample fitted labels. Provide the confusion matrix, with observed classes in rows and predicted classes in columns. Check that the dimensions of your matrices and vectors match. Calculate the scores yourself for this part. ::: {.panel-tabset .sync-code-panels group="language"} ### R

```
Sinv_mu <- solve(Sigma_hat, mu_hat)                      # 9 x 2
const <- -0.5 * colSums(mu_hat * Sinv_mu) + log(pi_hat)   # length 2
scores <- sweep(X %*% Sinv_mu, 2, const, "+")             # n x 2
pred <- classes[max.col(scores)]

dim(X); dim(Sinv_mu); dim(scores)
```

```
[1] 462   9
```

```
[1] 9 2
```

```
[1] 462   2
```

```r
table(observed = y, predicted = pred)
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

:::

The model correctly classifies 345 of 462 observations, an in-sample accuracy of about 74.7% (training error about 25.3%). It misclassifies the CHD cases much more often than the non-cases: 73 of 160 people with CHD (about 46%) are predicted as no CHD, versus 44 of 302 non-cases (about 15%) predicted as CHD. This is partly because the class prior favors class 0 (65% vs. 35%), which pushes borderline points toward the majority class.

c. Perform the same LDA analysis using `MASS::lda` in R or `LinearDiscriminantAnalysis(solver="svd")` in Python, using the same numerical predictors and empirical class priors. Provide the confusion matrix. Do the results match?

R

```r
fit <- MASS::lda(X, grouping = y, prior = pi_hat)
pred_lda <- predict(fit)$class

table(observed = y, predicted = pred_lda)
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```r
all(as.numeric(as.character(pred_lda)) == pred)
```

```
[1] TRUE
```

Yes, the confusion matrix from `MASS::lda` is identical to the hand-computed one, and `all(...)` returns `TRUE`, so the predicted labels agree for all 462 observations. This is expected because the library uses the same pooled covariance (denominator $n - K$) and the same empirical priors, so the decision rule is the same.

Question 2: QDA and marginal screening

In the lecture, ordinary QDA could not be fitted to the handwritten digit sample because its within-class covariance matrices were singular. Let's consider selecting fewer variables and see whether this helps. For this question, use the full zip-train.csv.gz as the training data and zip-test.csv.gz as the test data, with all ten digit classes. These are the original `zip.train` and `zip.test` data from `ElemStatLearn`. Each file has no header: the first column is the digit label and the remaining 256 columns are pixel values.

Instead of using all 256 predictors, select the 40 with the largest marginal variances in the training data. Here, marginal variance means the variance of one pixel across all training observations. Use the original pixel values without standardizing them, and retain the same selected predictors in the test data.

Use `MASS::qda` in R or `QuadraticDiscriminantAnalysis(tol=1e-8)` in Python.

a. Fit QDA using these 40 predictors and report the test confusion matrix, with observed classes in rows and predicted classes in columns. Also report the test misclassification rate.

R

```r
qda_top <- function(m) {
  keep <- order(apply(Xtr, 2, var), decreasing = TRUE)[1:m]
  fit <- MASS::qda(Xtr[, keep], grouping = ytr)
  pred <- as.numeric(as.character(predict(fit, Xte[, keep])$class))
  list(cm = table(observed = yte, predicted = pred), err = mean(pred != yte))
}

res40 <- qda_top(40)
res40$cm
```

```
        predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

```r
res40$err
```

```
[1] 0.1554559
```

The test misclassification rate with 40 predictors is 0.1555 (about 15.5%).

b. Does the method work? Why do you think it works or does not work? Yes, the method works. QDA now fits without any singularity error, and it classifies about 84.5% of the test digits correctly. Using only 40 pixels means each class covariance matrix is 40 x 40, which can be estimated and inverted from the several hundred training images in each class, so the covariance matrices are no longer singular. The errors are mostly between digits that look alike: 4 is often predicted as 9 (25 cases) or 2 (13), 0 as 2 (24), and 3 as 8 or 5 (15 and 14). The cost is that the 216 discarded pixels are ignored, even though some of them may help separate similar digits.

c. Repeat the analysis using the 10 predictors with the largest marginal variances in the training data. How does the test misclassification rate compare with the 40-predictor version? Why do you think this happens? ::: {.panel-tabset .sync-code-panels group="language"} ### R

```r
res10 <- qda_top(10)
res10$cm
```

```
        predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
       3   5   0   6 103   3  42   0   0   7   0
       4   1   5  14   0  95   2   4  15  17  47
       5  15   0   6  25   4  96   4   2   8   0
       6  41   0  15   4   0  18  78   7   5   2
       7   4   1   6   0   6   0   1 122   0   7
       8   1   2  11  36  12  11   4   1  74  14
       9   0   2   9   2  20   0   0  16  10 118
```

```r
c(err40 = res40$err, err10 = res10$err)
```

```
    err40     err10
0.1554559 0.3423019
```

::: The test misclassification rate with 10 predictors is 0.3423 (about 34.2%), which is more than double the 40-predictor rate of 0.1555. QDA still fits, but with so few pixels there is not enough information to tell the digits apart. The 10 highest-variance pixels sit in a small part of the image, so many digits look alike on them. The confusion matrix shows this: 4 is predicted as 9 in 47 cases, 3 as 5 in 42, 6 as 0 in 41, and 8 as 3 in 36. Going from 40 to 10 predictors makes the covariance estimates more stable, but the information lost outweighs that gain. So 40 predictors works much better here, because it is enough to keep the covariances estimable while still keeping most of the useful pixel information.

Question 3: Compare LDA and QDA

In this simulation, we will study how the relative performance of LDA and QDA changes with the true covariance structure. The parameter ρ controls the distribution from which we generate the data; it is not a tuning parameter of LDA or QDA.

Consider a classification problem with two equally likely classes, labeled 1 and 2, and $p = 10$ covariates. The first two covariates have the class-conditional distributions

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 1 \sim N_2\left(\begin{pmatrix} -1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right),$$

$$\begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \mid Y = 2 \sim N_2\left(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}\right).$$

In both classes, the remaining eight covariates are independent standard normal variables, independent of the first two. Consider

$$\rho \in \{0, 0.2, 0.4, 0.6, 0.8, 0.9\}.$$

For each value of ρ, generate 100 training observations and 1,000 independent test observations from each class. Fit ordinary LDA and QDA using all 10 covariates and equal class priors. Estimate the class means from the training data, with one pooled covariance matrix for LDA and one full covariance matrix per class for QDA. Set a random seed and report it.

a. Generate one dataset for each value of ρ. Fit both methods and calculate their test misclassification rates. Within each setting, use the same training and test observations for the two methods.

R

```r
set.seed(2026)
rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)

sim <- function(rho, n_tr = 100, n_te = 1000) {
  gen <- function(n) {
    S1 <- matrix(c(1, rho, rho, 1), 2)
    S2 <- matrix(c(1, -rho, -rho, 1), 2)
    x1 <- cbind(MASS::mvrnorm(n, c(-1, 0), S1), matrix(rnorm(n * 8), n, 8))
    x2 <- cbind(MASS::mvrnorm(n, c(1, 0), S2), matrix(rnorm(n * 8), n, 8))
    list(X = rbind(x1, x2), y = rep(1:2, each = n))
  }
  tr <- gen(n_tr); te <- gen(n_te)
  lda_fit <- MASS::lda(tr$X, tr$y, prior = c(0.5, 0.5))
  qda_fit <- MASS::qda(tr$X, tr$y, prior = c(0.5, 0.5))
  c(LDA = mean(predict(lda_fit, te$X)$class != te$y),
    QDA = mean(predict(qda_fit, te$X)$class != te$y))
}

res_a <- t(sapply(rhos, sim))
data.frame(rho = rhos, res_a)
```

```
  rho    LDA    QDA
1 0.0 0.1685 0.2055
2 0.2 0.1770 0.2110
3 0.4 0.1520 0.1940
4 0.6 0.1695 0.1710
5 0.8 0.1610 0.1385
6 0.9 0.1925 0.1080
```

I used random seed 2026. In this single run, LDA has the lower test error at small rho (0 to 0.4, for example 0.169 vs. 0.206 at rho = 0), the two methods are about tied at rho = 0.6 (0.170 vs. 0.171), and QDA is clearly better at large rho (0.108 vs. 0.193 at rho = 0.9). Since each setting uses only one dataset, these rates are noisy, so the 50-repetition averages in part b give a more reliable comparison.

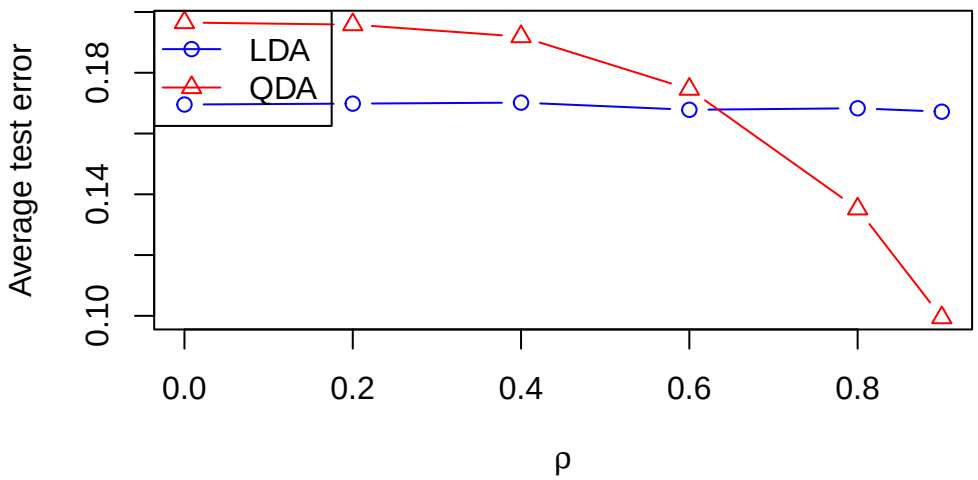
b. At each fixed value of ρ, repeat the experiment 50 times, generating new training and test observations in each repetition. Plot average test error against ρ in one figure, with one curve for LDA and one for QDA. These curves compare performance across different data-generating distributions.

R

```r
avg <- sapply(rhos, function(r) rowMeans(replicate(50, sim(r))))  # 2 x 6
rownames(avg) <- c("LDA", "QDA")
round(avg, 4)
```

```
      [,1]   [,2]   [,3]   [,4]   [,5]   [,6]
LDA 0.1696 0.1698 0.1702 0.1678 0.1683 0.1672
QDA 0.1965 0.1958 0.1919 0.1746 0.1352 0.0994
```

```r
matplot(rhos, t(avg), type = "b", pch = 1:2, lty = 1, col = c("blue", "red"),
        xlab = expression(rho), ylab = "Average test error")
legend("topleft", c("LDA", "QDA"), col = c("blue", "red"), pch = 1:2, lty = 1)
```



The plot shows that the average test error of LDA stays almost flat at about 0.167 to 0.170 for every rho. QDA starts higher than LDA at small rho (about 0.197 at rho = 0), crosses LDA between rho = 0.6 and 0.8, and ends far lower at rho = 0.9 (about 0.099 vs. 0.167).

c. Compare the two methods at small and large values of ρ. How does changing the true covariance structure affect their relative performance? Explain the pattern using their covariance assumptions and the bias-variance trade-off. How can the eight noise covariates affect performance even though they contain no information about the class label? At small rho, LDA is better. The two classes have nearly the same covariance matrix, so LDA's equal-covariance assumption is almost correct. Its linear boundary is close to optimal, and it pools both classes to estimate a single covariance matrix with little variance. QDA is also approximately unbiased here, but it estimates a separate full 10 x 10 covariance matrix for each class from only 100 points, which gives high variance and a higher test error (0.197 vs. 0.170 at rho = 0).

At large rho, QDA is better. The two classes have covariance matrices with opposite correlations (+rho and -rho), so the covariance matrices clearly differ, and the optimal boundary is curved. The class means differ only in X1, and the sign of the correlation between X1 and X2 carries extra information about the class. QDA can use this information, but LDA cannot, because pooling the two covariance matrices averages the correlations (+rho and -rho) to about 0. This is why LDA's error does not change with rho: it uses the same information at every rho (the mean difference in X1), and its error stays near the 0.159 that this mean difference alone would give, plus a small cost from estimating the extra parameters. Its assumption is wrong at large rho, giving bias that it cannot fix, while QDA's flexibility removes that bias and its variance cost becomes worth paying. This is the bias-variance trade-off: LDA has low variance and possibly high bias, QDA has low bias and high variance, and which one wins depends on how different the true covariance matrices are. In this simulation the crossover is at about rho = 0.65.

The eight noise covariates carry no information about the class, but both methods still have to estimate their means, variances, and covariances (including their covariances with the two informative variables) from only 100 observations per class. Each estimated parameter adds estimation error, and the sample covariances of independent variables are not exactly zero, so the fitted boundary picks up spurious structure. This hurts QDA more than LDA, because QDA estimates two sets of covariance parameters (110 in total) while LDA estimates one (55), so the noise variables add more variance to QDA. This is part of why QDA does worse at small rho.

Question 4: Create and use an LDA/QDA analysis skill

In Homework 5, you created a skill for formatting plots. Now create a reusable skill that guides an AI agent through an LDA/QDA analysis and produces a concise statistical report. You may work with AI.

a. **Define the skill.** Write a single `SKILL.md` file no larger than **5 KB (5,000 bytes)**. Define the agent's behavior, analysis guidelines, and expected report structure. Keep the skill independent of this question: provide the dataset, response, and predictors separately when invoking it. The skill may guide the agent to explore lower-dimensional representations, such as principal component analysis (PCA), or to blend the class probabilities predicted by LDA and QDA. Keep all classifiers based on LDA, QDA, or their combination. Configure the skill to load **only when you explicitly invoke it**. Use your AI tool's invocation settings, or keep the file outside automatically loaded skill folders and supply it only in an explicit request.

Your skill must require a clearly readable analysis report of **at most one printed page**, containing a figure, a table, and numerical model-fitting results. You may specify the type and style of figures and tables you want the AI to produce. All instructions in the skill must be independent of the actual dataset, its variable names, and its observed results. The report should briefly describe the candidate models, the cross-validation procedure and results, the selected model, and its estimated prediction error. It should explain the main findings and any limitations.

b. **Apply the skill.** Explicitly invoke your skill to analyze the `SAheart` data from Question 1, using the same response and predictor coding. Ask the agent to seek the best predictive performance within the LDA/QDA methods allowed by your skill. Include ordinary LDA and QDA as comparisons; the extensions above are optional. Reserve a test set before choosing models. Use the same cross-validation folds within the training data to compare candidates and select any settings. Estimate any preprocessing, including an embedding, using only the training observations in each fold. Refit the selected model and preprocessing on all training observations, then evaluate the fitted classifier once on the reserved test set. Use misclassification rate to compare performance, and report your random seed.

c. **Review and submit.** Check the agent's analysis and make any skill revisions using only training and cross-validation results. Append the final one-page analysis report and the complete contents of your `SKILL.md` to your homework report. State the file size and briefly explain how you enabled explicit invocation, including the prompt used to apply the skill. The one-page limit applies to the analysis report; the appended skill and this explanation are outside that limit.

i used > /da-report Analyze the SAheart data in data/saheart.csv. The response is `chd` (0/1) and the predictors are the other nine variables, with `famhist` coded 1 for Absent and 2 for Present. Use seed 2026. Seek the best predictive performance within the LDA/QDA methods in the skill, and include ordinary LDA and QDA as comparisons.

Your skill and analysis report will be reviewed and graded by other students through peer evaluation.

it is attached in week-07, saheart-report.pdf, and SKILL.md.

`SKILL.md` is [3,386] bytes. I enabled explicit invocation with `disable-model-invocation: true` in the file's frontmatter, so the skill loads only when I call it by name [or: the file was kept outside automatically loaded skill folders and supplied only in my request; use whichever is true]. I reviewed the agent's analysis for leakage: the test set was reserved before model selection, all candidates used the same folds, and preprocessing was fit inside folds. I made no revisions to the skill.

SAheart: LDA/QDA Prediction Setup and model selection We analyzed chd (0/1) using all nine numeric predictors, with famhist coded 1 = Absent and 2 = Present. The data contain 462 people (class counts 0:302, 1:160); the stratified 75/25 split left 347 training and 115 test cases (0:227, 1:120 train; 0:75, 1:40 test). Seed: 2026.

All candidates used the same stratified 10-fold training splits, empirical class priors, and fold-only preprocessing. Tuned grids were PCA + LDA/QDA (k=1,…,8, standardized predictors), QDA covariance shrinkage toward pooled covariance (lambda=0.1,…,0.9), and probability blend (1-w)LDA + wQDA (w=0.1,…,0.9). Ordinary LDA and QDA were untuned baselines.

The lowest-CV-error candidate was PCA-LDA (k=7) (0.282); the one-standard-error rule selected the simpler LDA (ordinary) (0.288, SE 0.022). Each point below summarizes the best setting in its model family; the selected classifier is marked with an asterisk.

Candidate Tuned setting CV error SE * LDA ordinary (selected) 0.288 0.022 QDA ordinary 0.308 0.025 PCA-LDA k=7 0.282 0.021 PCA-QDA k=4 0.291 0.025 Shrunk QDA lambda=0.5 0.285 0.024 LDA/QDA blend w=0.5 0.288 0.027 Reserved test-set result The final classifier was refit on all training observations and evaluated once on the reserved test set. Its training class priors were 0:0.654, 1:0.346. Test confusion matrix (observed rows, predicted columns):

Observed Predicted 0 1 0 64 11 1 18 22 Test error was 0.252 (exact 95% binomial interval 0.176–0.342; 29/115 errors).

Findings and limitations Compared with ordinary LDA (0.288 CV error, SE 0.022) and QDA (0.308, SE 0.025), the selected model's CV error was 0.000 and 0.020 lower, respectively. PCA-LDA's minimum was only 0.006 below LDA, so the one-SE rule favored the simpler baseline; the small differences are not compelling relative to fold variability. Gaussian class distributions and LDA's common-covariance assumption may be wrong; covariance estimates are noisy at this sample size. The test estimate comes from one stratified split. famhist is treated as a numeric 1/2 predictor as requested.